Task 13 - Secure API Testing & Authorization Validation

CHAPTER 1: Introduction to REST APIs

An Application Programming Interface (API) is a set of rules that allows different software applications to communicate with each other. APIs act as an intermediary between a client and a server, enabling data exchange in a structured and controlled manner. In modern applications, APIs are widely used to connect web applications, mobile apps, and backend services.

REST (Representational State Transfer) is a popular architectural style used for designing APIs. REST APIs are based on standard HTTP protocols and are designed to be simple, scalable, and stateless. Each request from a client to the server contains all the necessary information to process the request, without relying on stored session data.

REST APIs play a crucial role in modern software development as they enable seamless integration between systems. However, because APIs expose application functionality and data, they are also a common target for security attacks. This makes secure API design and testing essential to prevent unauthorized access, data leakage, and misuse of services.

CHAPTER 2: HTTP Methods & Status Codes

HTTP methods define the type of action a client wants to perform on an API resource. Each method has a specific purpose and is used to interact with server-side data in a controlled manner. Understanding HTTP methods is essential for both API development and security testing, as misuse of methods can lead to unintended access or data modification.

The most used HTTP methods in REST APIs include:

- **GET** – Used to retrieve data from the server. It is a read-only operation and should not modify server-side data.
- **POST** – Used to send data to the server, often to create new resources or perform authentication.
- **PUT** – Used to update existing resources with new data.
- **DELETE** – Used to remove resources from the server.

In addition to HTTP methods, APIs communicate the outcome of a request using HTTP status codes. Status codes help clients understand whether a request was successful or why it failed.

Common HTTP status codes observed during API security testing include:

- **200 OK** – The request was successful, and the server returned the expected response.
- **400 Bad Request** – The request contained invalid or malformed data.
- **401 Unauthorized** – Authentication is required, or the provided credentials are invalid.
- **403 Forbidden** – The server understood the request but refused to authorize it.
- **404 Not Found** – The requested resource does not exist.
- **429 Too Many Requests** – The client has sent too many requests in a short period, indicating rate limiting.

Analysing HTTP methods and status codes helps security analysts identify improper access control, authentication issues, and error-handling weaknesses in APIs.

CHAPTER 3: API Authentication Concepts

Authentication is the process of verifying the identity of a client or user attempting to access an API. It ensures that only authorized users or systems can interact with protected API endpoints. Without proper authentication, APIs may be exposed to unauthorized access, leading to data breaches and service misuse.

APIs commonly use different authentication mechanisms depending on the application's security requirements. Some widely used authentication methods include:

- **Basic Authentication** – Involves sending a username and password with each request, typically encoded in the Authorization header. Although simple, it must be used over secure channels such as HTTPS.
- **Token-Based Authentication** – Uses tokens (such as bearer tokens or API keys) that are issued after successful login. These tokens are included in subsequent requests to validate access without repeatedly sending credentials.

During API security testing, authentication mechanisms are tested by sending requests with valid credentials, invalid credentials, and missing authentication information. Properly

secured APIs should reject unauthorized requests and return appropriate error codes such as 401 Unauthorized or 403 Forbidden.

Understanding API authentication concepts helps security analysts identify weaknesses related to improper credential handling and access control.

CHAPTER 4: API Authorization & Access Control

Authorization determines what actions an authenticated user is allowed to perform within an API. While authentication verifies the identity of a user, authorization controls access to specific resources and operations based on permissions. Proper authorization ensures that users can only access data and functionality they are permitted to use.

One of the most common API security issues related to authorization is **Broken Object Level Authorization (BOLA)**, also known as **Insecure Direct Object Reference (IDOR)**. This occurs when an API allows users to access or manipulate resources by modifying object identifiers, such as user IDs or resource IDs, without proper permission checks.

During API security testing, authorization is evaluated by modifying resource identifiers in API requests and observing whether unauthorized data is exposed. Secure APIs should enforce access control checks and deny access to resources that the requester is not authorized to view or modify.

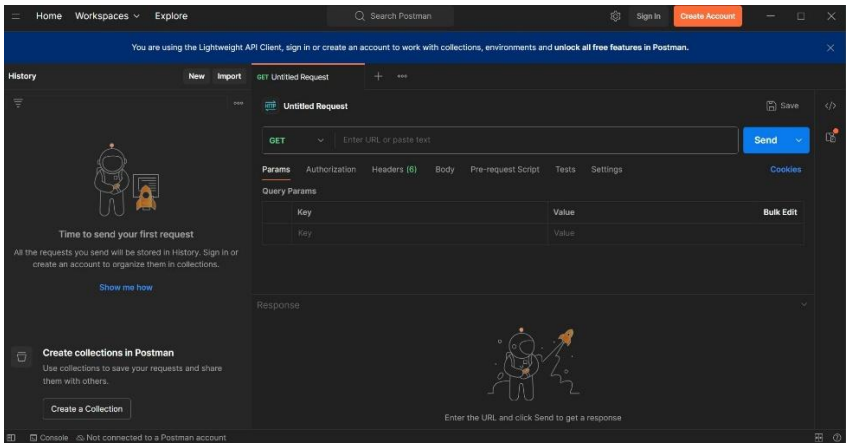
Understanding authorization and access control mechanisms is critical for preventing unauthorized data access, privilege escalation, and data leakage in API-driven applications.

CHAPTER 5: Tools Used for API Security Testing

API security testing requires tools that allow testers to send requests, modify parameters, analyse responses, and observe API behavior in a controlled manner. Selecting the right tools helps ensure accurate testing while maintaining ethical and safe practices.

Postman was used as the primary tool for this task. Postman provides a user-friendly interface to create and send HTTP requests, configure authentication headers, and analyse API responses. It supports various HTTP methods such as GET and POST and displays response codes, headers, and body content clearly, making it suitable for manual API security testing.

Figure 5.1: Postman tool used for API security testing



(Postman was used as the primary tool to create and send API requests, analyze responses, and perform manual API security testing.)

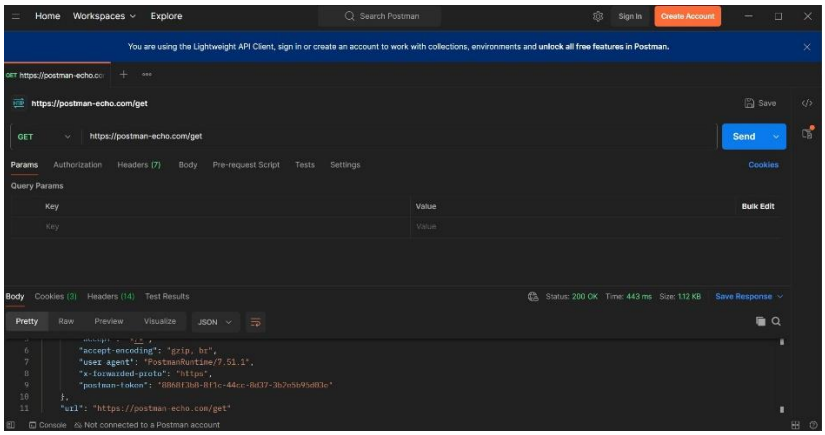
In addition to Postman, other tools are commonly used in API testing environments. These include **cURL**, a command-line tool used to send HTTP requests, and **Insomnia**, a lightweight API client like Postman. These tools were not used practically in this task but are relevant in real-world API testing scenarios.

Using Postman allowed controlled and ethical testing of public APIs while focusing on understanding authentication, authorization, input validation, and response behavior.

CHAPTER 6: Secure API Testing Methodology

A structured and ethical testing methodology is essential for effective API security assessment. In this task, a **manual and controlled testing approach** was followed to analyze API behavior without causing disruption or misuse. The scope of testing was limited to **publicly available test APIs**, ensuring that no real user data or production systems were affected.

Figure 6.1: API request configuration in Postman



(API requests were manually configured in Postman to test authentication, authorization, input validation, and response behavior in a controlled environment.)

The testing process began with understanding the API endpoints and their expected functionality. Requests were sent using different HTTP methods to observe normal API behavior. Authentication mechanisms were then tested by providing valid credentials, invalid credentials, and by removing authentication headers to evaluate access control enforcement.

Authorization testing was performed by modifying resource identifiers in API requests to assess whether access restrictions were properly enforced. Input validation was tested by submitting malformed and unexpected data to observe how the API handled invalid input. Additionally, repeated requests were sent manually to observe any rate-limiting or abuse-prevention mechanisms.

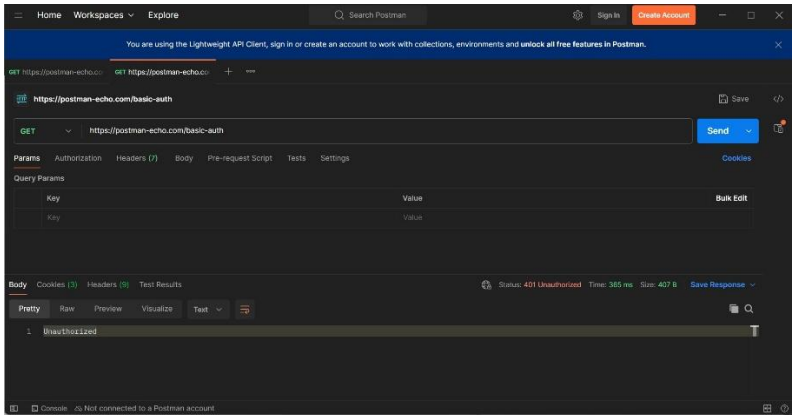
Throughout the testing process, API responses, status codes, and error messages were carefully reviewed to identify potential security weaknesses. All observations were documented, and no automated attacks or exploitation techniques were used, maintaining an ethical and educational testing environment.

CHAPTER 7: Authentication Testing Analysis

Authentication testing focuses on verifying whether an API correctly enforces identity validation before granting access to protected resources. Proper authentication ensures that only legitimate users or systems can interact with sensitive API endpoints.

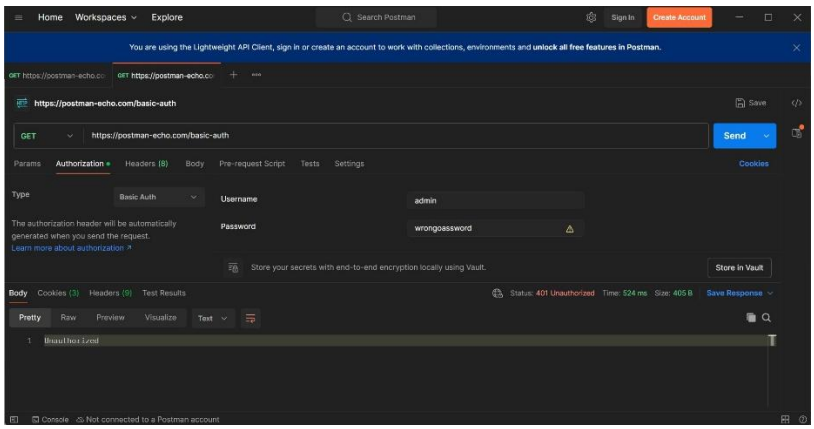
In this task, authentication behavior was tested by sending requests with **missing credentials**, **invalid credentials**, and by explicitly **removing authentication headers**. When authentication information was not provided or incorrect credentials were supplied, the API consistently returned error responses such as **401 Unauthorized** or **403 Forbidden**. This indicates that the API correctly validates authentication data and prevents unauthorized access.

Figure 7.1: Authentication failure when credentials are missing



(The API rejected unauthenticated requests by returning an appropriate error response, indicating proper authentication enforcement.)

Figure 7.2: API rejecting invalid authentication credentials



(Requests with incorrect authentication credentials were denied, demonstrating effective credential validation.)

The consistent rejection of unauthenticated and improperly authenticated requests demonstrates effective authentication enforcement. Additionally, error messages returned by the API did not disclose sensitive information, such as user existence or internal system details, which helps reduce the risk of information leakage.

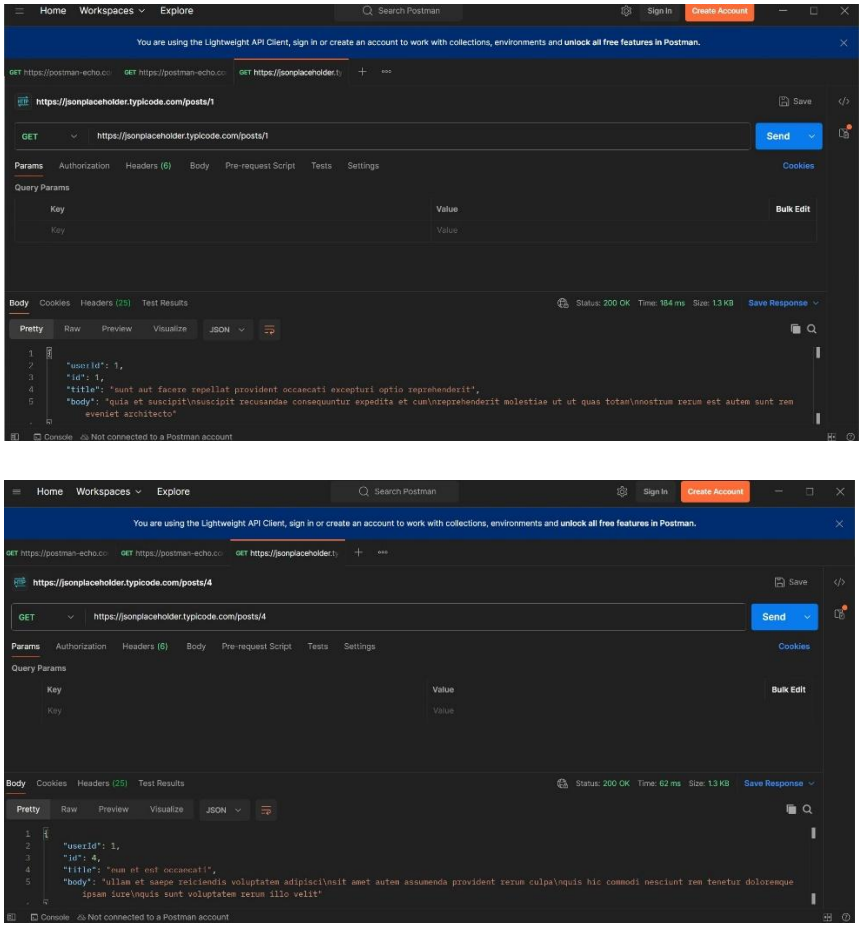
Through authentication testing, the importance of enforcing strict access control and validating credentials at every request was clearly observed.

CHAPTER 8: Authorization Testing & IDOR Analysis

Authorization testing evaluates whether an API properly restricts access to resources based on user permissions. Even after successful authentication, users should only be able to access data and actions that they are authorized for. Weak authorization controls can lead to serious security issues such as data exposure and privilege escalation.

One common authorization vulnerability in APIs is **Broken Object Level Authorization (BOLA)**, also known as **Insecure Direct Object Reference (IDOR)**. This occurs when an API allows access to resources by simply modifying object identifiers, such as user IDs or resource IDs, without verifying whether the requester has permission to access those resources.

Figure 8.1: Resource identifier manipulation for authorization testing (IDOR)



(Resource identifiers were modified to test authorization controls. The test API returned data as it is publicly accessible by design.)

In this task, authorization testing was performed by modifying resource identifiers within API requests and observing the responses. The test API used was publicly accessible by design, and therefore returned data for different resource identifiers. While this behavior is expected for public APIs, similar behavior in authenticated or sensitive APIs could result in unauthorized data access.

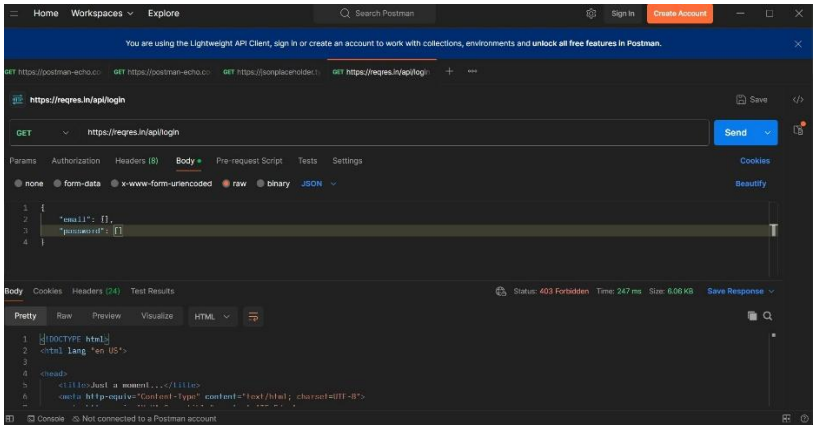
This analysis highlights the importance of implementing strict authorization checks on the server side to ensure that users can only access resources they are permitted to view or modify.

CHAPTER 9: Input Validation & Error Handling

Input validation is a critical aspect of API security that ensures only properly formatted and expected data is processed by the server. APIs must validate input data types, required fields, and value constraints to prevent unexpected behavior, crashes, or security vulnerabilities.

In this task, input validation was tested by sending **malformed and invalid request payloads** to the API endpoint. The API correctly rejected the invalid input and returned appropriate error responses such as **403 Forbidden** or **400 Bad Request**, depending on how the backend enforced validation and access control.

Figure 9.1: Input validation test using malformed request payload



(The API safely rejected malformed input data without exposing sensitive error details, indicating proper input validation and secure error handling.)

Proper error handling was observed, as the API did not expose sensitive information such as stack traces, database errors, or internal system details in its responses. Secure error messages help prevent attackers from gaining insights into backend logic or system architecture.

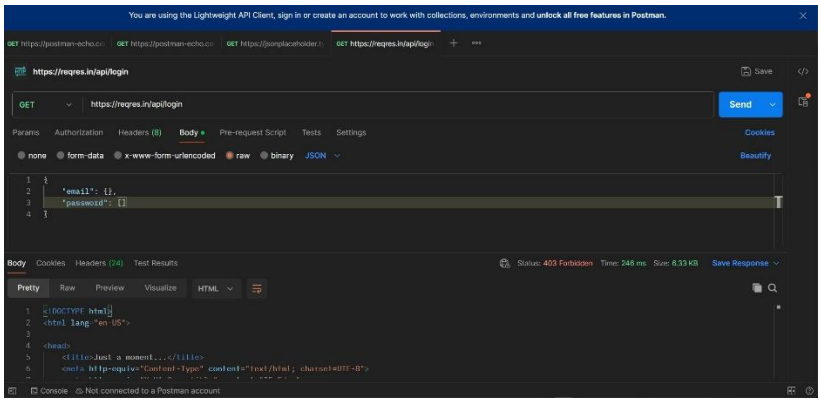
This testing demonstrates the importance of strict input validation and secure error handling in protecting APIs against injection attacks, data corruption, and information disclosure.

CHAPTER 10: Rate Limiting & Abuse Prevention

Rate limiting is a security mechanism used to control the number of requests a client can send to an API within a specified time period. It helps protect APIs from abuse scenarios such as brute-force attacks, credential stuffing, denial-of-service attempts, and excessive automated requests.

In this task, rate limiting behavior was tested by sending multiple consecutive requests manually to the same API endpoint. The API consistently returned a **403 Forbidden** response for unauthorized requests, indicating that access was blocked at the authorization stage. No distinct rate-limiting response such as **429 Too Many Requests** was observed during testing.

Figure 10.1: Rate limiting observation using repeated API requests



(Multiple consecutive requests were sent to observe rate limiting behaviour. The API consistently blocked unauthorized requests without exposing rate-limit details.)

This behavior suggests that rate limiting may be enforced only after successful authentication or may not be explicitly implemented for unauthorized requests in the test API. While public test APIs may allow such behavior by design, production APIs should implement proper rate-limiting controls to prevent abuse and automated attacks.

Understanding rate limiting and abuse prevention mechanisms is essential for designing resilient and secure APIs.

CHAPTER 11: OWASP API Security Risk Mapping

The OWASP API Security Top 10 identifies the most critical security risks affecting modern APIs. Mapping practical testing results to these risks helps security professionals understand the real-world impact of vulnerabilities and prioritize mitigation efforts.

During this task, the following OWASP API security risks were studied and conceptually mapped based on observed API behavior:

- **Broken Object Level Authorization (BOLA):**
  Authorization testing involved modifying resource identifiers to assess whether access controls were enforced. Although the test API was publicly accessible by design, similar behavior in protected APIs could lead to unauthorized data access.

- **Broken Authentication:**
  Authentication testing demonstrated that missing or invalid credentials were consistently rejected. Proper enforcement of authentication mechanisms reduces the risk of unauthorized access.

- **Improper Input Validation:**
  Malformed request payloads were rejected by the API, indicating input validation controls. Weak input validation in real-world APIs can lead to injection attacks and unexpected system behavior.

- **Security Misconfiguration:**
  Consistent error handling and the absence of sensitive information in API responses indicate proper configuration. Misconfigured APIs may expose internal details or sensitive data through error messages.

By mapping testing observations to OWASP API risks, this task highlights how structured API security testing helps identify potential weaknesses and reinforces the importance of secure API design and implementation.

CHAPTER 12: Security Observations & Findings

This chapter summarizes the key observations made during the API security testing process. The findings are based on manual testing performed using Postman on publicly available test APIs and focus on authentication, authorization, input validation, and response behavior.

Authentication testing showed that the API correctly enforced access controls by rejecting requests with missing or invalid credentials. Unauthorized requests consistently returned appropriate HTTP status codes such as 401 Unauthorized and 403 Forbidden, indicating proper authentication handling.

Authorization testing using resource identifier manipulation demonstrated expected behavior for a public test API. While the API returned data for modified identifiers by design, the test highlighted how similar behavior in protected APIs could lead to Broken Object Level Authorization (IDOR) vulnerabilities if proper access checks are not implemented. Input validation testing confirmed that malformed and unexpected input was safely rejected. The API returned controlled error responses without exposing sensitive information such as stack traces or internal system details, demonstrating secure error handling practices. Rate limiting tests showed that unauthorized requests were blocked at the authorization level, and no explicit rate-limiting response was observed. This behavior is common in test APIs and emphasizes the importance of implementing rate-limiting mechanisms in production environments to prevent abuse.

Overall, the API demonstrated secure behavior in handling authentication, authorization, and error responses, with no critical security exploitation performed during testing.

CHAPTER 13: Conclusion & Learnings

This task provided hands-on exposure to secure API testing using a structured and ethical approach. By testing authentication, authorization, input validation, rate limiting behavior, and response handling, the task highlighted how APIs enforce access control and protect against common security risks. Through practical testing using Postman, key API security concepts such as authentication enforcement, authorization checks, and secure error handling were clearly understood. Mapping the observed behavior to OWASP API Security risks further reinforced the importance of secure API design and standardized security practices. The task also emphasized that effective API security testing does not require aggressive exploitation techniques. Careful observation of API responses, status codes, and behavior is sufficient to identify potential weaknesses and assess security posture.

Overall, this task strengthened understanding of real-world API security testing workflows, improved familiarity with industry tools, and reinforced the importance of ethical and responsible security assessment practices.